Constructor University

School of Computer Science and Engineering

Compiling Kotlin Coroutines to WebAssembly with Stack-switching

by

Veronika Sirotkina

a thesis for conferral of a Master of Science in Advanced Software Technology

First reviewer: Anton Podkopaev
Second reviewer: Ezra Cooper
Industry supervisor: Zalim Bashorov

May 19, 2026

Abstract

This thesis presents an alternative implementation of Kotlin Coroutines for Kotlin/Wasm using the WebAssembly Stack-switching proposal, replacing the current state-machine transformation with direct Stack-switching instructions.

The main technical challenge is a mismatch between the two models: Kotlin requires a continuation object to be available before a function suspends, while Wasm Stack-switching only provides it after suspension. The thesis describes two implementations that resolve this issue.

The Stack-switching implementation reduces binary size by approximately 10.8% in production builds and 2.3% in development builds. Performance benchmarks on a production VM (V8) show a regression: realistic cases are around 2–2.5 times slower than the baseline, and cases with intensive suspension chaining are 4.5–5.7 times slower without a special V8 flag, or around 1.7 times slower with it. The Stack-switching proposal is still under active development, and the performance may improve as both the Kotlin Coroutines and V8 implementations mature.

Acknowledgments

I would like to thank Zalim Bashorov for guiding me through the initial development process and helping me navigate the compiler code. I would also like to thank Aleksandr Shefer, who took over the implementation and is now bringing it to production.

Contents

List of Figures

List of Tables

Listings

Chapter 1

Introduction

1.1 Context

WebAssembly (Wasm) is a binary instruction format for a stack-based virtual machine [2]. Its most mainstream use case is in browsers. Many languages can be compiled to it, including Kotlin. Kotlin/Wasm is used, for example, by the Compose Multiplatform library [3] for its Web target. Compose Multiplatform also uses Kotlin Coroutines extensively in its implementation.

The current implementation of Kotlin Coroutines for Kotlin/Wasm is based on the implementation for Kotlin/JS, which, similar to Kotlin/JVM, uses state-machine transformation on suspend functions. Each suspend function is compiled into a state machine. This transformation adds generated code to the binary and introduces overhead at runtime.

Wasm has a proposal called Stack-switching. It introduces new primitives for suspending and resuming execution contexts at the Wasm level, which can be used to write an alternative implementation of Kotlin Coroutines for Kotlin/Wasm without state-machine transformation.

1.2 Goal

The goal of this thesis is to implement Kotlin Coroutines for Kotlin/Wasm using WebAssembly Stack-switching primitives, and to evaluate whether this implementation improves binary size and performance compared to the existing state-machine-based approach.

A central technical challenge is the mismatch between Kotlin's coroutine model, where a continuation object must be available before a suspend function actually suspends, and the Wasm Stack-switching model, where the continuation only becomes available after suspension. This thesis describes two implementations that address this

mismatch and then evaluates their correctness and the effect they have on resulting binary size and performance.

1.3 Thesis Structure

The thesis is organized as follows: Chapter 2 provides background information on WebAssembly Stack-switching and Kotlin coroutines primitives. Chapter 4 describes the implementation of Kotlin coroutines using the WebAssembly Stack-switching proposal. Chapter 5 evaluates the implementation in terms of correctness, binary size, and performance. Finally, Chapter 6 concludes the thesis and discusses the current state of the coroutines with Stack-switching implementation.

Chapter 2

Background

2.1 Kotlin Coroutines

2.1.1 Introduction

The Kotlin standard library provides a set of low-level coroutine primitives that define how coroutines are created, suspended, and resumed. These primitives are the building blocks on top of which higher-level abstractions are implemented. The `kotlinx.coroutines` library, for example, provides structured concurrency constructs such as `launch`, `async`, and `Flow` by using these primitives internally.

2.1.2 Coroutines example

Figure 2.1 shows an example program built on the low-level coroutine primitives. The program starts a coroutine. After the coroutine suspends, it is resumed with the value "OK" and the completion saves the result of the coroutine to a variable. The result is then printed.

```kotlin
var continuation: Continuation<String>? = null
var coroutineResult: String? = null

suspend fun suspendFun(): String {
    val result = suspendCoroutineUninterceptedOrReturn { cont ->
        continuation = cont
        COROUTINE_SUSPENDED
    }
    return result
}

object Completion : Continuation<String> {
    override val context: CoroutineContext = EmptyCoroutineContext
    override fun resumeWith(result: Result<String>) {
        coroutineResult = result.getOrThrow()
    }
}

fun main() {
    // returns COROUTINE_SUSPENDED
    ::suspendFun.startCoroutineUninterceptedOrReturn(Completion)
    continuation.resume("OK")
    println(coroutineResult) // prints "OK"
}
```

Figure 2.1: Kotlin coroutines primitives example

2.1.3 Primitives

The following sections describe the relevant coroutine primitives from the standard library.

`Continuation` and `CoroutineImpl`

`Continuation<T>` represents a coroutine at some point in its execution. That point could be the beginning of the coroutine if it was not started yet, or some point at which the coroutine was interrupted by calling `suspendCoroutineUninterceptedOrReturn` function. Resuming continuation resumes the coroutine from the recorded point. A continuation can be understood as "the rest of a coroutine." The interface has two main components:

- `context: CoroutineContext` – the context of the coroutine
- `resumeWith(result: Result<T>)` - resumes the coroutine execution, passing a result value that becomes the return value of the `suspendCoroutineUninterceptedOrReturn` call

This interface is implemented by class `CoroutineImpl`. `CoroutineImpl` is a partial implementation of a coroutine that implements some logic for processing resuming and

handling exceptions.

Suspend functions

Suspend functions in Kotlin represent a computation that can be suspended and resumed later.

Every suspend function receives an additional parameter `completion` of type `Continuation`. This completion is resumed after the suspend function completes. This means that whenever you see that a function has type `suspend () -> T`, the actual type of the function after a particular lowering during compilation will become `(completion: Continuation<T>) -> T`.

`startCoroutineUninterceptedOrReturn`

This primitive starts a coroutine from the given suspend function:

```
(suspend () -> T).startCoroutineUninterceptedOrReturn(
    completion: Continuation<T>
): Any?
```

`startCoroutineUninterceptedOrReturn` immediately starts the suspend function with the given `completion` continuation as the completion parameter (remember, suspend functions are appended an additional completion parameter during compilation). The `completion` parameter is the continuation that will be resumed when the coroutine finishes its execution (either with a result or an exception). The function `startCoroutineUninterceptedOrReturn` returns:

- `COROUTINE_SUSPENDED` if the coroutine suspends
- The result value if the coroutine completes immediately without suspension

`createCoroutineUnintercepted`

This primitive creates a coroutine without starting it:

```
(suspend () -> T).createCoroutineUnintercepted(
    completion: Continuation<T>
): Continuation<Unit>
```

`suspendCoroutineUninterceptedOrReturn`

This primitive gives access to the current continuation inside a suspend function. It allows the function to control whether it suspends:

```
suspendCoroutineUninterceptedOrReturn<T>(
    block: (Continuation<T>) -> Any?
): T
```

The `block` receives the current continuation and can:

- Return `COROUTINE_SUSPENDED` to indicate that the coroutine has suspended and will be resumed later
- Return a value without suspension

2.1.4 Internal behavior

To understand how these primitives interact with each other internally, see Figure 2.2. `startCoroutineUninterceptedOrReturn` primitive creates a coroutine object `CoroutineImpl` from a suspend function and resumes it. Internally, the `CoroutineImpl::resumeWith` method calls the `doResume` method, which contains the actual logic of resuming the underlying suspend function.

At some point in the execution of a suspend function, a `suspendCoroutineUninterceptedOrReturn` primitive may be encountered. This primitive procures a continuation object which represents the rest of the current coroutine and passes it to the suspend lambda. Suspend lambda is a block of user code, which is meant to save the continuation somewhere so that it can be resumed later in case the coroutine suspends. Suspend lambda also dictates whether the coroutine actually suspends. If suspend lambda returns `COROUTINE_SUSPENDED`, the coroutine will suspend; otherwise, the execution will continue. If the coroutine suspends, it can only be resumed by resuming the continuation object that was previously passed to the suspend lambda.

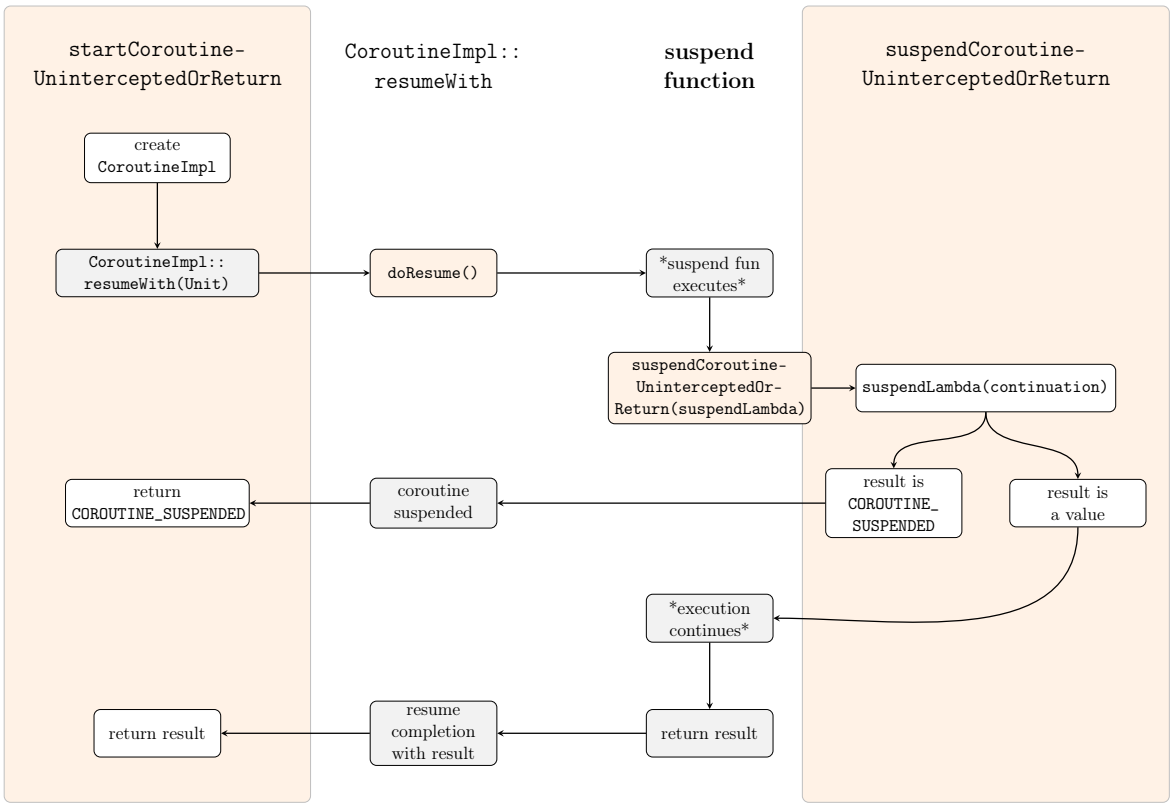


Figure 2.2: Kotlin Coroutine primitives internals

Orange blocks and columns represent functions that need to be implemented

White blocks represent logic that needs to be implemented inside unimplemented functions

Gray blocks represent calls to functions and logic that don't need to be implemented or require minimal changes

2.2 WebAssembly and Stack-switching

2.2.1 Intuition

Stack-switching proposal introduces a continuation type and instructions to work with it. Unlike Kotlin's `Continuation<T>`, which is typed by a single resume value type `T`, a Wasm continuation is typed by the full signature of the underlying function. Both Kotlin and Wasm continuations are *one-shot*: a continuation may be resumed at most once.

A good intuition for understanding resuming and suspending in the Stack-switching proposal is that it is similar to calling a function inside a try-catch block: the `resume` instruction (try block) specifies handlers (catch blocks) for different *suspension tags* (exception types). A function resumed via continuation can suspend with a `suspend`

instruction. Depending on the suspension tag provided with the `suspend` instruction, execution will jump to a specific suspension handler. That is where differences from throwing an exception appear: after jumping to the handler, there will be a new continuation object on the stack. Resuming this continuation will resume the function that has just suspended. This is in contrast to throwing exceptions: throwing an exception ends the function completely and it can't be resumed.

In Figure 2.3 you can see the most important Stack-switching primitives and what they do.

2.2.2 Primitives

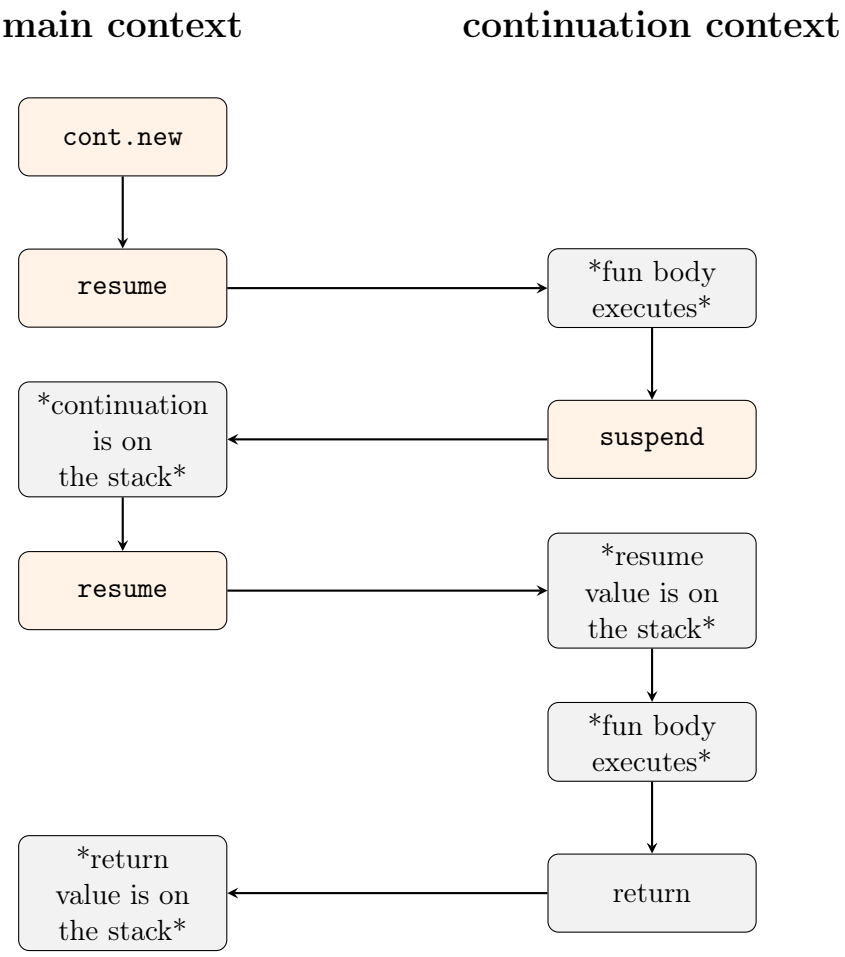


Figure 2.3: Stack-switching primitives

Below is an introduction for each of the depicted primitives.

`cont.new`

`cont.new` instruction creates a continuation from a function. Continuation has a type that reflects the signature of the underlying function.

`resume`

`resume` instruction resumes a continuation and specifies what to do when the underlying function suspends by installing suspension handlers for particular suspension tags. As you may recall from the beginning of the section, continuation objects have a type that matches the underlying function. Since functions can have arguments, continuations can also expect some values to be passed to fill those arguments. The `resume` instruction expects these values to be on the stack.

`suspend`

`suspend` instruction suspends the execution with some suspension tag. Suspension tags also have a function-like type. The input parameters of a suspension tag are values that will be taken by `suspend` instruction from the stack and passed to the suspension handler. The result type of a suspension tag describes the values that should appear on the stack of the suspended function after it's resumed again. It also becomes the input type of the continuation object that will appear on the stack at the handler. Essentially, it is possible to pass values both from the main context into continuation and from continuation into the main context, and suspension tags define the types of the passed values.

```
;; Tag used to coordinate between generator and consumer:
;; The i32 param corresponds to the generated values passed
;; to consumer; no values are passed from consumer to generator.
(tag $generate (param i32))

(func $generator
 (local $i i32)
 (local.set $i (i32.const 100))
 (loop $loop
  ;; Suspend execution, pass current value of $i to consumer.
  (suspend $generate (local.get $i))
  ;; Decrement $i and exit loop once $i reaches 0.
  (local.tee $i (i32.sub (local.get $i) (i32.const 1)))
  (br_if $loop)
 )
)

(func $consumer
 (local $generator_cont (ref $cont_type))
 ;; Create continuation executing function $generator.
 (local.set $generator_cont (cont.new $cont_type (ref.func $generator)
    ))
 (loop $loop
  (block $on_generate (result i32 (ref $cont_type))
   ;; Resume continuation $generator_cont.
   ;; When $generate tag is thrown,
   ;; go to label $on_generate (points to right after this block)
   (resume $cont_type (on $generate $on_generate) (local.get $
      generator_cont))
   ;; $generator returned: no more data.
   (return)
  )
  ;; Generator suspended, stack now contains [i32 (ref $cont_type)]
  ;; Save continuation to resume it in next iteration
  (local.set $generator_cont)
  ;; Stack now contains the i32 value yielded by $generator
  (call $print)
  ;; Go to the beginning of the loop (br means branch)
  (br $loop)
 )
)
```

Figure 2.4: Stack-switching example in WebAssembly (adapted from an example from the proposal repository [1])

Figure 2.4 gives an example code using Stack-switching in Wasm. This is a simple program with generator and consumer. Consumer creates a continuation from generator and continuously resumes it and prints the received values. Generator produces values from 100 down to 1 and passes them to the consumer via the suspend instruction. If the generator is resumed but all the values have already been passed, it returns. Consumer sees that the generator has finished and also returns.

2.2.3 State of the proposal

As of today, the Stack-switching proposal is at Phase 3 (Implementation Phase) of the WebAssembly standardization process [4]. At this phase the proposal has a complete specification text and implementations in WebAssembly runtimes. Currently, the most notable implementation is the one used in V8, the JavaScript and WebAssembly engine used by Chromium-based browsers, including Google Chrome. The proposal is being actively developed with the goal of eventually adding it to the WebAssembly standard.

Chapter 3

Related Work

This chapter discusses approaches related to compiling coroutines and asynchronous code to WebAssembly, as well as alternative designs for Stack-switching in Wasm.

3.1 Approaches to Asynchronous Code in WebAssembly

3.1.1 Asyncify

Before native stack-switching support, one of the most widely used techniques for supporting asynchronous-style code in WebAssembly was Asyncify [5], a binary transformation pass implemented in Binaryen. Asyncify works by rewriting a compiled Wasm binary to make it possible to pause and resume execution at arbitrary points. At suspension points, the transformation inserts code to unwind the current call stack into a pre-allocated buffer, and at resumption points it inserts code to rewind the stack from that buffer.

The approach is analogous in motivation to the state-machine transformation used by the Kotlin compiler for the JVM and JS targets: both enable suspension and resumption without native stack-switching support. However, Kotlin performs a source/IR-level stackless coroutine lowering, whereas Asyncify instruments the generated Wasm binary to save and restore stack state. The key difference is that Asyncify operates at the Wasm binary level rather than at the source or IR level, making it language-agnostic.

Asyncify has been used by several language runtimes and ports, including Python (Pyodide) and some Ruby-on-Wasm efforts. Its main drawbacks are increased binary size and runtime overhead from the stack save and restore operations [5].

3.1.2 JS Promise Integration (JSPI)

An alternative approach for asynchronous WebAssembly is the JS Promise Integration (JSPI) proposal [6]. JSPI provides a JavaScript API that allows Wasm imports and exports to be wrapped so that a synchronous Wasm call can suspend waiting for a JavaScript `Promise` to settle, and then be resumed when the promise resolves. From the Wasm binary's perspective, JSPI can be transparent: unlike Asyncify, it does not require rewriting the module, although the relevant imports and exports must be wrapped by the JavaScript host. The suspension and resumption are managed by the JavaScript engine.

JSPI addresses the specific use case of interoperating with asynchronous JavaScript APIs from synchronous Wasm code. JSPI does not expose general-purpose stack-switching primitives within the Wasm module itself. Therefore, by itself, it is not a suitable mechanism for implementing arbitrary language-level coroutines that suspend and resume entirely within Wasm. The Stack-switching proposal is a more general solution that covers this use case as well.

JSPI has been presented and discussed in the WebAssembly Stack Subgroup [7]. It has also been partially implemented in SpiderMonkey using Stack-switching primitives [8].

3.1.3 WasmFX and Typed Continuations

WasmFX [9] is an alternative proposal for adding stack-switching capabilities to WebAssembly, based on the theory of algebraic effects and typed continuations. WasmFX also uses typed continuations, with core operations such as `cont.new`, `resume`, and `suspend`, and additional constructs such as `cont.bind`, `resume_throw`, and `barrier`. The core difference from the Stack-switching proposal is that WasmFX continuations are designed to compose with effect handlers, enabling a richer set of control-flow abstractions.

WasmFX was presented and developed in the WebAssembly Stack Subgroup over several meetings [10, 11]. A prototype implementation in the Wasmtime engine was discussed in the subgroup in 2023 [12]. The current Stack-switching proposal has evolved alongside WasmFX/typed-continuation work, and the two lines of work have influenced discussions in the Stack Subgroup.

3.2 Other Language Runtimes on WebAssembly

Multiple language communities have explored how to compile their runtimes to WebAssembly, which requires solving problems similar to the ones addressed in this thesis.

Go and TinyGo. The Go runtime uses goroutines, lightweight runtime-managed threads with dynamically managed stacks. Two separate presentations in the WebAssembly Stack Subgroup discussed the challenges of compiling Go to Wasm: one covering the full Go runtime [13], and one covering TinyGo [14], a Go compiler for embedded and Wasm targets. Both noted the difficulty of mapping Go's concurrency model to WebAssembly's linear stack model.

OCaml. Multicore OCaml adds support for algebraic effects, which generalize coroutines and other control-flow abstractions. The OCaml community presented its experience compiling to Wasm in the Stack Subgroup [15], discussing how native OCaml effects could be compiled to WebAssembly using stack-switching primitives.

Erlang. Erlang's language model is based on lightweight isolated processes with their own execution state and message passing. Challenges in implementing the Erlang runtime in WebAssembly were presented in the Stack Subgroup [16], noting that native Wasm stack switching could simplify such ports compared with approaches based on binary instrumentation such as Asyncify.

These cases, together with Kotlin, illustrate that language-level concurrency abstractions and stack-switching support in WebAssembly are broadly needed across many language communities.

Chapter 4

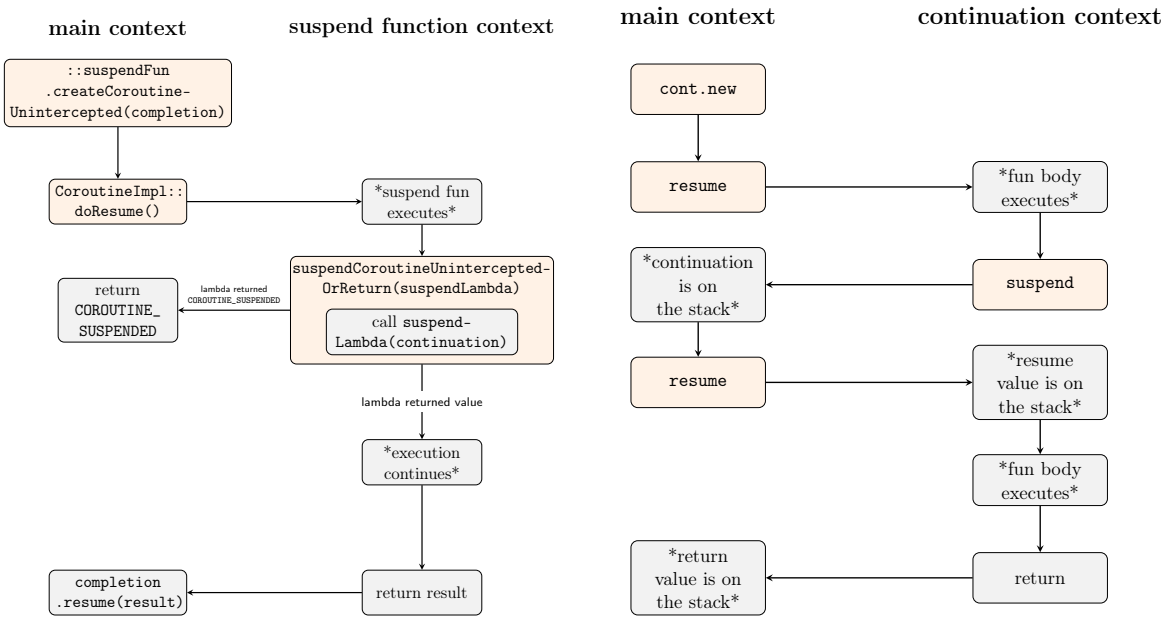
Implementation

4.1 Primitives correspondence

To implement Kotlin coroutines using the Wasm Stack-switching proposal, the primitives discussed in Section 2.1 must be implemented. Kotlin coroutine primitives map closely to the Wasm Stack-switching primitives:

- `Continuation` maps to Wasm continuation objects.
- `createCoroutineUnintercepted` maps to `cont.new`.
- `startCoroutineUninterceptedOrReturn` maps to `cont.new` followed by `resume`.
- `suspendCoroutineUninterceptedOrReturn` maps to the `suspend` instruction with some additional logic to execute suspend lambda (the `block` argument that is passed to `suspendCoroutineUninterceptedOrReturn`).
- `Continuation::resumeWith` maps to the `resume` instruction.

This correspondence can be observed more closely in Figure 4.1.



(a) Kotlin coroutine primitives (b) Wasm Stack-switching primitives

Figure 4.1: Comparison between Kotlin and Wasm coroutine primitives

Unfortunately, there is one crucial difference in how suspension is handled in Kotlin, which makes it not so obvious how to implement it in Wasm. In Kotlin the function `suspendCoroutineUninterceptedOrReturn` accepts a lambda which either returns some result or suspends the execution. This lambda accepts an argument: current continuation. Resuming this continuation will resume the function. The intention of providing this argument is for the suspension lambda to contain code that saves the continuation in some variable. Then, in case suspension really happens, the function will be resumed by resuming this saved continuation.

For example, consider this simple suspend function, that adheres to the intention behind suspension lambdas:

```
1 var savedContinuation: Continuation<String>? = null
2
3 suspend fun suspendHere(): String =
4     suspendCoroutineUninterceptedOrReturn { cont ->
5         // Save continuation before suspending
6         savedContinuation = cont
7         // Indicate suspension occurred
8         COROUTINE_SUSPENDED
9     }
```

In this code, the continuation `cont` is available inside the lambda and can be saved to `savedContinuation` before the function actually suspends. Later, calling `savedContinuation?.resume("result")` will resume the suspended function.

However, there is a more subtle case that must also be supported: the continuation can be resumed immediately inside the suspension lambda. Although this pattern is

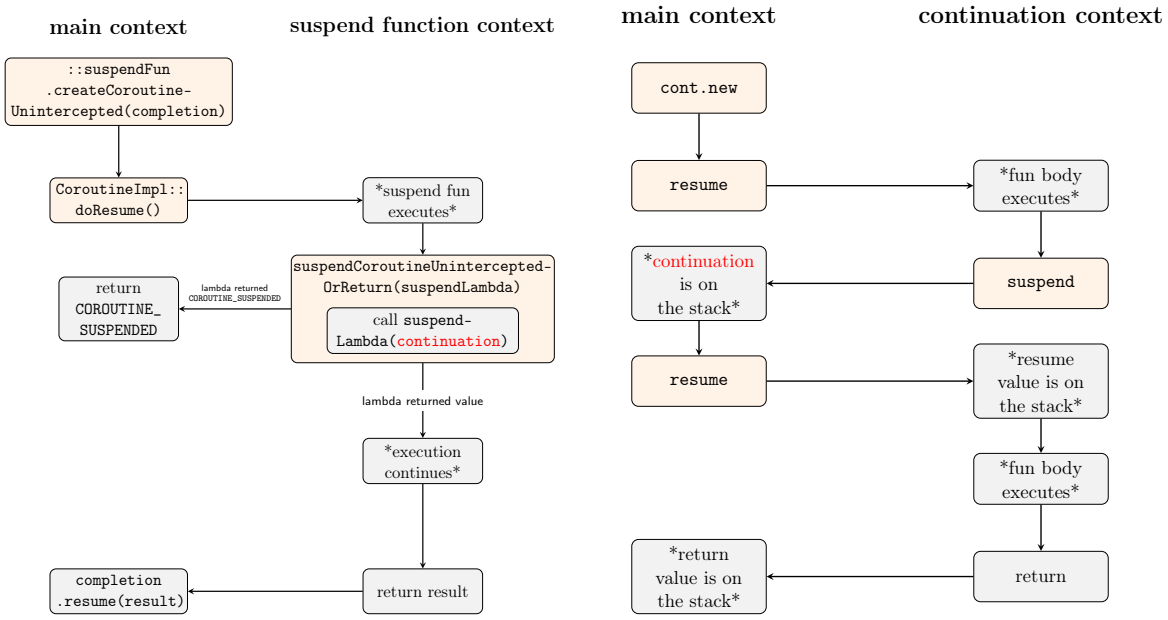
unusual in user code, it is not prohibited and is covered by Kotlin's semantics tests. The pattern is illustrated by the following example:

```
suspend fun suspendFun(): String =
    suspendCoroutineUninterceptedOrReturn { cont ->
        cont.resume("OK")       // Resume immediately
        COROUTINE_SUSPENDED     // Still indicate suspension
    }

fun test() {
    var completionResult: String? = null
    val result = suspendFun.startCoroutineUninterceptedOrReturn(
        object : Continuation<String> {
            override val context = EmptyCoroutineContext
            override fun resumeWith(value: Result<String>) {
                completionResult = value.getOrThrow()
            }
        }
    )
    // result == COROUTINE_SUSPENDED
    // completionResult == "OK"
}
```

In Kotlin, the continuation object must be available before it is known whether a function suspends. In Wasm, the continuation only becomes available after the function actually suspends. This mismatch is illustrated in Figure 4.2.

The following sections describe two ways to resolve this problem.



(a) Kotlin coroutine primitives (b) Wasm Stack-switching primitives

Figure 4.2: Continuation availability problem between Kotlin and Wasm

4.2 First implementation

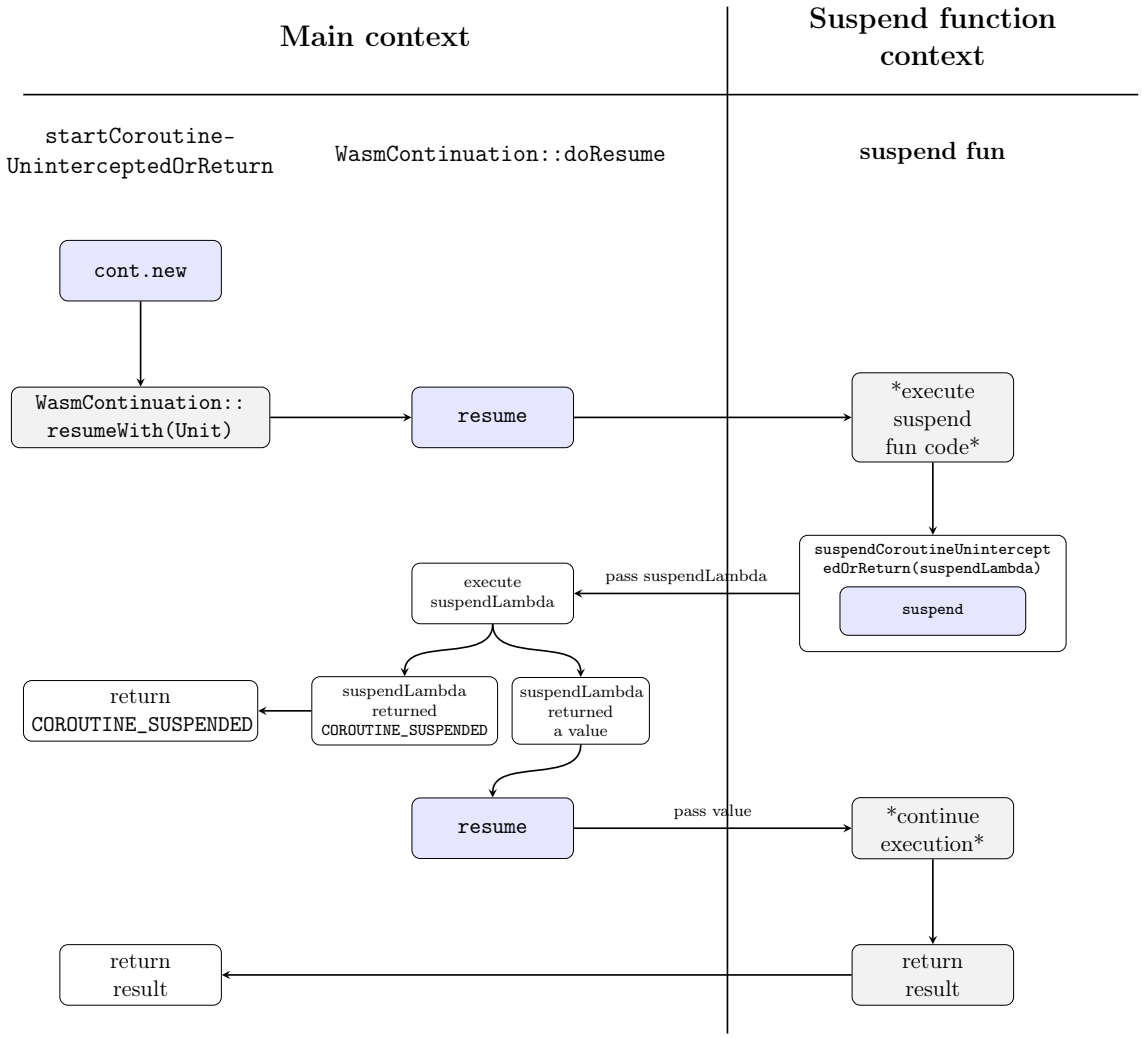


Figure 4.3: First implementation idea

The first implementation idea is shown in Figure 4.3. The control flow is as follows.

`startCoroutineUninterceptedOrReturn` is called on a suspend function. The first thing we do is create a Wasm continuation from the suspend function.

`WasmContinuation` is a class that inherits from `CoroutineImpl` and represents a coroutine that works on a Wasm continuation object. The one method in `CoroutineImpl` that needs to be implemented in `WasmContinuation` is `CoroutineImpl::doResume()`. When we call `WasmContinuation::resumeWith`, it in turn calls `doResume`, which contains the core logic for resuming a coroutine.

In `doResume` we call the Wasm instruction to resume the continuation. The type of Wasm continuations handled in `WasmContinuation` is always the same: they have to be resumed with an object of type `Any?`. When `WasmContinuation` is resumed in `startCoroutineUninterceptedOrReturn`, the resume value will be a completion. As discussed in Section 2.1.3, completion is a parameter added to all suspend functions

during compilation. When `WasmContinuation` is resumed outside `startCoroutineUninterceptedOrReturn`, the resume value will be the value provided with the method `resumeWith`

When a suspend function is being executed, it may at some point call `suspendCoroutineUninterceptedOrReturn`. `suspendCoroutineUninterceptedOrReturn` has one argument that we will call suspend lambda. Suspend lambda needs to be executed to decide whether we should suspend or not, but to do so it requires that we pass it a continuation. This continuation should represent the rest of the current suspend function. In the context of the suspend function that's currently being executed, we can't provide the necessary continuation. Instead, we can suspend, and with `suspend` instruction we can pass the lambda that needs to be executed to the main context, where the Wasm continuation we need becomes available.

The `suspend` instruction puts the execution back into the body of `doResume`. At the top of the stack there will be a suspend lambda to be executed and the Wasm continuation object we need for it. So, the next step is to create a `WasmContinuation` from the Wasm continuation and execute the suspend lambda with it. If the lambda returns `COROUTINE_SUSPENDED`, that means that the coroutine really suspended and it's time to return. If the lambda returns another value, that means that we have to keep executing the suspend function. So, we resume the new continuation with the value returned from the lambda.

This solves the problem of immediate resume, because at the moment when `suspendLambda` is called, the suspension has already happened, so we can provide the `suspendLambda` with the actual Wasm continuation.

4.3 Immediate resume optimization

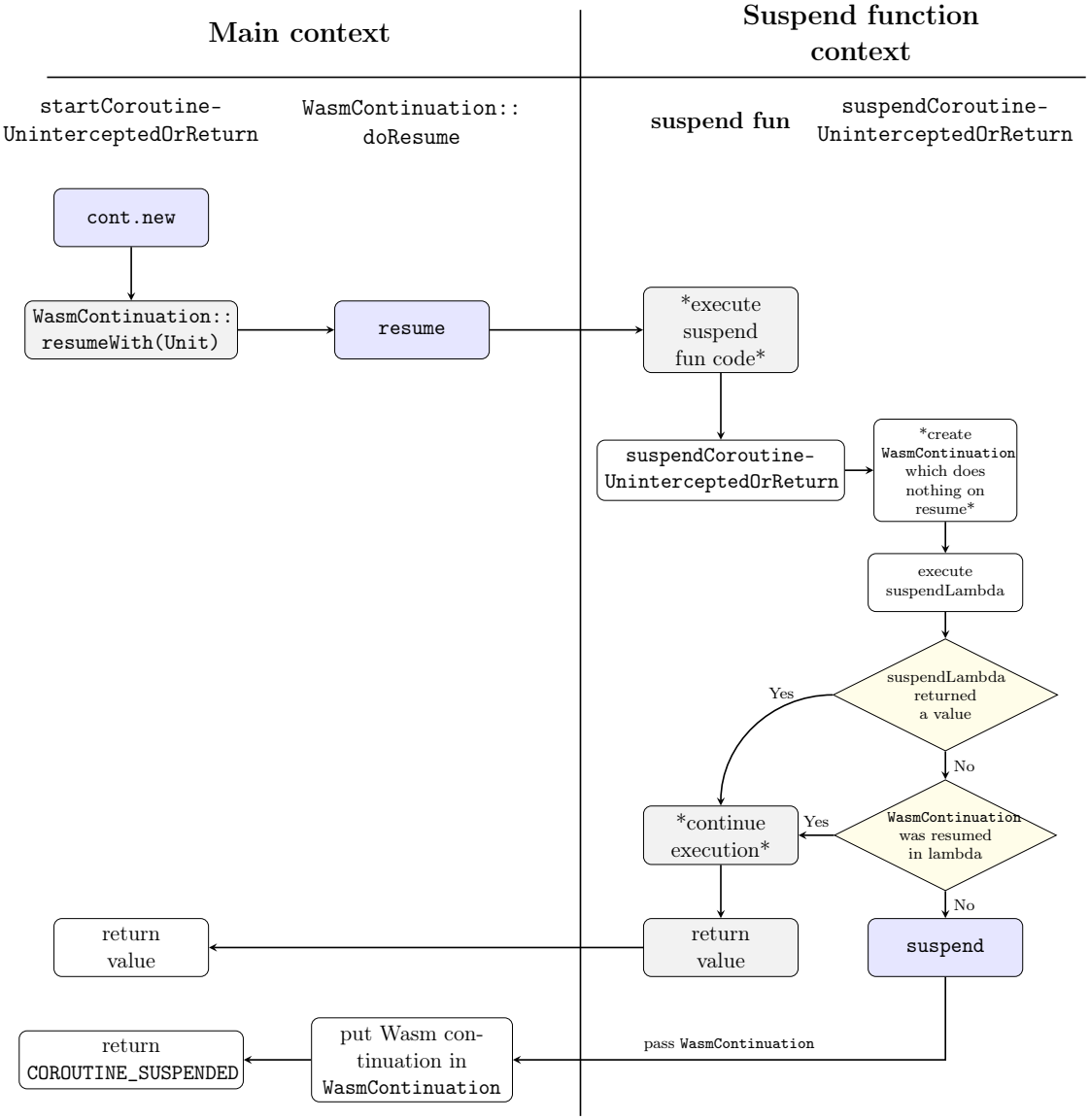


Figure 4.4: Immediate resume optimization

The optimized implementation idea is shown in Figure 4.4.

Behind this optimization is a heuristic. What is the actual meaning of resuming a continuation that is passed to the suspend lambda? It means that the suspension lambda requests immediate continuation of the suspended computation. With this in mind, we can adopt the following course of action: we call suspend lambda with a `WasmContinuation` object, that doesn't have an actual Wasm continuation inside. If the lambda saves this `WasmContinuation`, that is fine – we will tie it to a Wasm continuation later. If the lambda resumes this `WasmContinuation`, we will record the value passed to `resumeWith`, but otherwise we will do nothing. Once the lambda completes, we will check whether `WasmContinuation` was resumed, and if it was, we

will get the resume value out of it and act as if that was the value that the lambda returned.

Chapter 5

Evaluation

5.1 Correctness

Kotlin codebase already contains extensive tests for coroutines. We can separate them into two levels.

Intrinsic semantics tests

These tests verify the behavior of the lowest-level coroutine primitives like `startCoroutineUninterceptedOrReturn` and `suspendCoroutineUninterceptedOrReturn`. They also cover mid-level coroutine APIs built directly on top of them like `startCoroutine` and `suspendCoroutine`.

The tests are located in the Kotlin repository at `compiler/testData/codegen/box/coroutines/intrinsicSemantics`.

You can find an example of what kind of tests are in the `intrinsicSemantics` folder in Listing 1 in the Appendix 6).

All discussed Stack-switching implementations pass all intrinsic semantics tests.

Box tests for coroutines

This is an extensive collection of tests that verifies that coroutines work correctly when used together with other Kotlin features. Some of the things that are tested are:

- **Control flow** (tests in `controlFlow/` directory): testing coroutines with `if`, `while`, `try-catch-finally`, and other control flow structures.
- **Default and variadic parameters** (e.g., `defaultParametersInSuspend.kt`, `varargCallFromSuspend.kt`): testing suspend functions with default or variadic arguments.

- **Exception handling** (e.g., `handleException.kt`, `simpleException.kt`): testing that exceptions thrown inside coroutines are correctly passed to the completion continuation or handled by `try-catch` blocks.

As an example, below is a short test that verifies coroutine behavior with an `if` expression:

Listing 5.1: Example from controlFlow_if1.kt

```
suspend fun s1(): Int = suspendCoroutineUninterceptedOrReturn { x ->
    x.resume(42)
    COROUTINE_SUSPENDED
}

fun f1(): Int = 117
fun f2(): Int = 1
fun f3(x: Int, y: Int): Int = x + y

fun box(): String {
    var result = 0
    suspend {
        result = f3(if (f1() > 100) s1() else f2(), 42)
    }.startCoroutine(object : Continuation<Unit> {
        override val context = EmptyCoroutineContext
        override fun resumeWith(result: Result<Unit>) {}
    })

    if (result != 84) return "fail: $result"
    return "OK"
}
```

The tests are located in the Kotlin repository at `compiler/testData/codegen/box/coroutines`.

Both the initial and the optimized implementations pass 517 out of 523 coroutine tests. The 6 failing tests are caused by a bug in the implementation: in certain cases, the generated Wasm code omits explicit casts to `Any?`. In Kotlin, all types are subtypes of `Any?`, so no cast is needed; in Wasm, however, such casts must be emitted explicitly. Adding the missing casts fixes the failures. These tests do not exercise the coroutine suspension or resumption paths that are relevant to the size and performance analysis, so the failures do not affect the conclusions of the evaluation.

5.2 Size benchmark

To evaluate the impact of the Stack-switching implementation on binary size, a small Kotlin project generated by the Kotlin Multiplatform Wizard [17] was compiled using

three different compiler versions:

- **Baseline**: Kotlin 2.3.10 (original implementation without Stack-switching)
- **Stack-switching**: Base Stack-switching implementation rebased to Kotlin 2.3.10
- **Optimized SS**: Stack-switching with immediate resume optimization rebased to Kotlin 2.3.10

For each compiler version, two distributions were generated by running the following Gradle tasks:

- `wasmJsBrowserDevelopmentExecutableDistribution`: development build without optimizations
- `wasmJsBrowserDistribution`: production build with optimizations

Table 5.1 shows the size of the compiled `KotlinProject-composeApp.wasm` file in kilobytes for each configuration.

Table 5.1: Wasm binary size comparison (in KB)

Build Mode	Baseline	Stack-switching	Resume opt
Dev	20,460.67	19,987.97	19,992.46
Prod	2,082.82	1,857.17	1,858.34

Table 5.2 shows the percentage change in binary size compared to the baseline. Negative values indicate a size reduction.

Table 5.2: Size change compared to baseline (negative values indicate reduction)

Build Mode	Baseline	Stack-switching	Resume opt
Dev	20,460.67 KB	-2.3%	-2.3%
Prod	2,082.82 KB	-10.8%	-10.8%

The results show that both Stack-switching implementations achieve a reduction in binary size compared to the baseline. The size reduction is bigger in optimized builds (approximately 10.8%) compared to development builds (approximately 2.3%). The difference between the base Stack-switching implementation and the optimized version is negligible in terms of binary size, with both achieving similar reductions.

5.3 Performance benchmarks

The performance of the implementation was measured by running benchmarks on low-level coroutine primitives. The following cases are measured:

- `chainSuspensions.kt`: 20 coroutines are created and suspended. When the first coroutine is resumed, it suspends again, and in the suspend lambda the second coroutine is resumed. The second coroutine suspends and resumes the third coroutine in its suspend lambda. This continues on until the last coroutine is resumed, which records the fact that it was reached in a global variable. After that, all coroutines that were suspended so far are resumed so that they terminate properly.
- `deepSuspensions.kt`: same setup as in `chainSuspensions.kt`, but when the last coroutine in the chain is resumed, it also suspends and resumes 20 times.
- `multipleSuspensions1.kt`: create one coroutine and suspend and resume it 50 times. A pattern with immediate resume inside suspend lambda is used.
- `multipleSuspensions2.kt`: same as `multipleSuspensions1.kt`, but the immediate resume pattern is not used. For each suspension the continuation is saved and resumed from the main context.
- `sequence.kt`: this benchmark is an exception as it's not a benchmark on low-level primitives. It's a benchmark for a Kotlin sequence builder, which is built on top of coroutines. For this test a sequence builder is used to compute the first 20 numbers from the Fibonacci sequence.
- `startCoroutinePerformance.kt`: start 100 coroutines with `startCoroutine`.

Four code versions were used for comparison:

- **Baseline** (commit `359b8303`): The original implementation without Stack-switching
- **Stack-switching** (commit `96b5d449`): Initial Stack-switching implementation
- **Optimized SS** (commit `4ffd6205`): Stack-switching with immediate resume optimization
- **Opt + param** (commit `2f6a6ebb`): Optimized SS with `--experimental-wasm-growable-stacks` flag passed to V8

These commits refer to the Kotlin source repository [18].
The benchmarks were executed in two compilation modes:

- **dev mode**: Development build without optimizations
- **dce mode**: Production build with dead code elimination

All benchmarks were run on a MacBook Pro with an Apple M2 Max chip (12-core: 8 performance + 4 efficiency cores, 64 GB RAM) running macOS. The V8 version used was 14.7.89.

Each benchmark was run for 50 iterations after 50 warmup iterations. Some iterations were removed as outliers before computing the average: any measurement exceeding $Q_3 + 1.5 \times IQR$ (where Q_3 is the third quartile and IQR is the interquartile range) was excluded, as such spikes are typically caused by garbage collection pauses rather than the code under test. The bar charts in this chapter display the average value across the remaining iterations, with error bars showing the 95% confidence interval computed from the filtered sample using the t-distribution. The full benchmark data is available in the thesis repository at https://github.com/ozzush/kotlin-coroutines-stack-switching-thesis.

5.3.1 V8

V8 benchmark results are the most relevant, since V8 is the engine used by Chromium, and consequently Google Chrome, which is the most widely used browser [19].

Raw results

Tables 5.3 and 5.4 show the raw benchmark results for dev and dce modes respectively.

Table 5.3: Benchmark results in dev mode (average time in nanoseconds, V8 runtime)

Test	Baseline	Stack-switching	Optimized SS	Opt+param
chainSusp	18,644	93,227	96,872	30,711
deepSusp	23,119	118,298	102,826	42,617
multiSusp1	7,196	14,114	7,500	8,114
multiSusp2	6,930	14,395	16,000	16,439
sequence	2,250	9,214	11,935	9,809
startCor	25,020	37,625	42,562	32,842

Table 5.4: Benchmark results in dce mode (average time in nanoseconds, V8 runtime)

Test	Baseline	Stack-switching	Optimized SS	Opt+param
chainSusp	18,109	102,787	103,000	30,262
deepSusp	22,761	101,364	105,979	37,674
multiSusp1	6,947	13,447	7,043	7,163
multiSusp2	6,718	15,111	16,156	16,744
sequence	2,458	9,178	10,286	9,718
startCor	22,809	33,956	33,326	33,043

Percentage Changes

Table 5.5 presents the percentage changes relative to the baseline. Positive values indicate performance regression (slower execution), while negative values indicate improvement (faster execution).

Table 5.5: Percentage change compared to baseline (V8)

Test	Mode	Baseline	Stack-switching	Optimized SS	Opt+param
chainSusp	dev	18,644	+400.0%	+419.6%	+64.7%
deepSusp	dev	23,119	+411.7%	+344.8%	+84.3%
multiSusp1	dev	7,196	+96.1%	+4.2%	+12.8%
multiSusp2	dev	6,930	+107.7%	+130.9%	+137.2%
sequence	dev	2,250	+309.5%	+430.4%	+335.9%
startCor	dev	25,020	+50.4%	+70.1%	+49.5%
chainSusp	dce	18,109	+467.6%	+468.8%	+68.8%
deepSusp	dce	22,761	+345.3%	+365.6%	+65.5%
multiSusp1	dce	6,947	+93.6%	+1.4%	+3.1%
multiSusp2	dce	6,718	+124.9%	+140.5%	+149.2%
sequence	dce	2,458	+273.3%	+318.4%	+295.3%
startCor	dce	22,809	+48.9%	+46.1%	+44.9%

Visual Comparison

Figures 5.1 and 5.2 provide a visual comparison of the benchmark results across all three implementations.

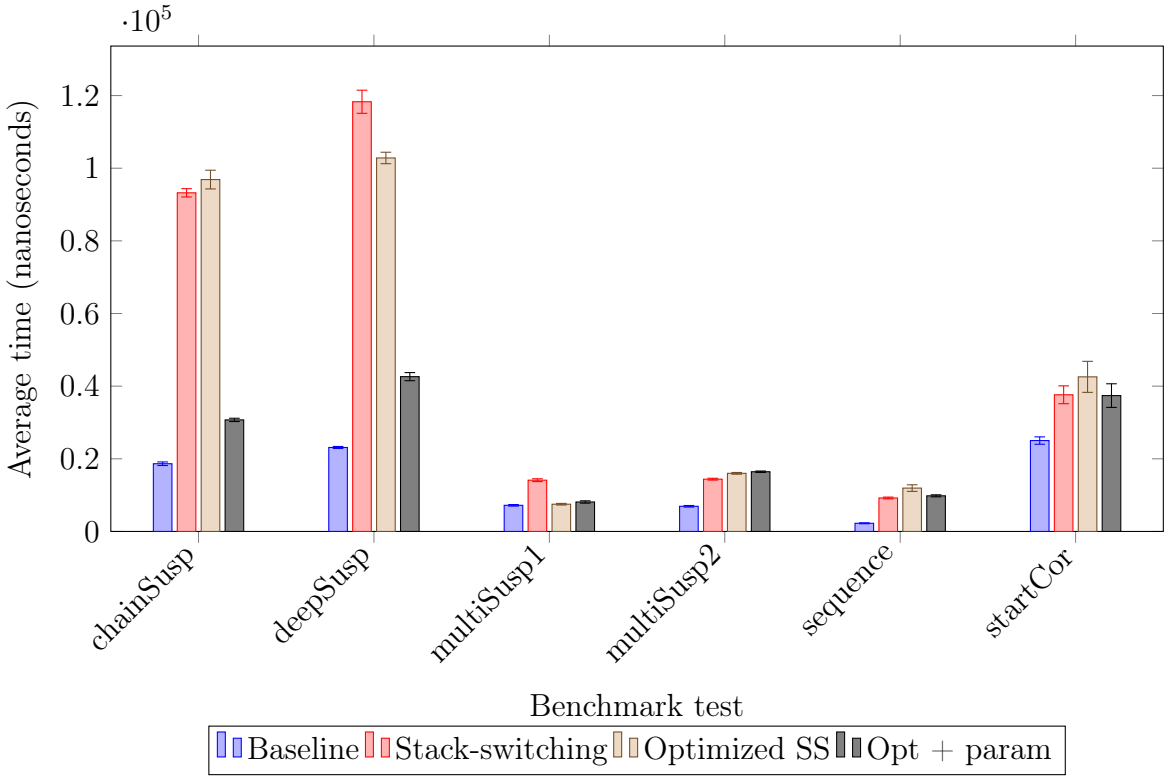


Figure 5.1: Benchmark comparison in dev mode, V8 runtime (error bars show 95% CI)

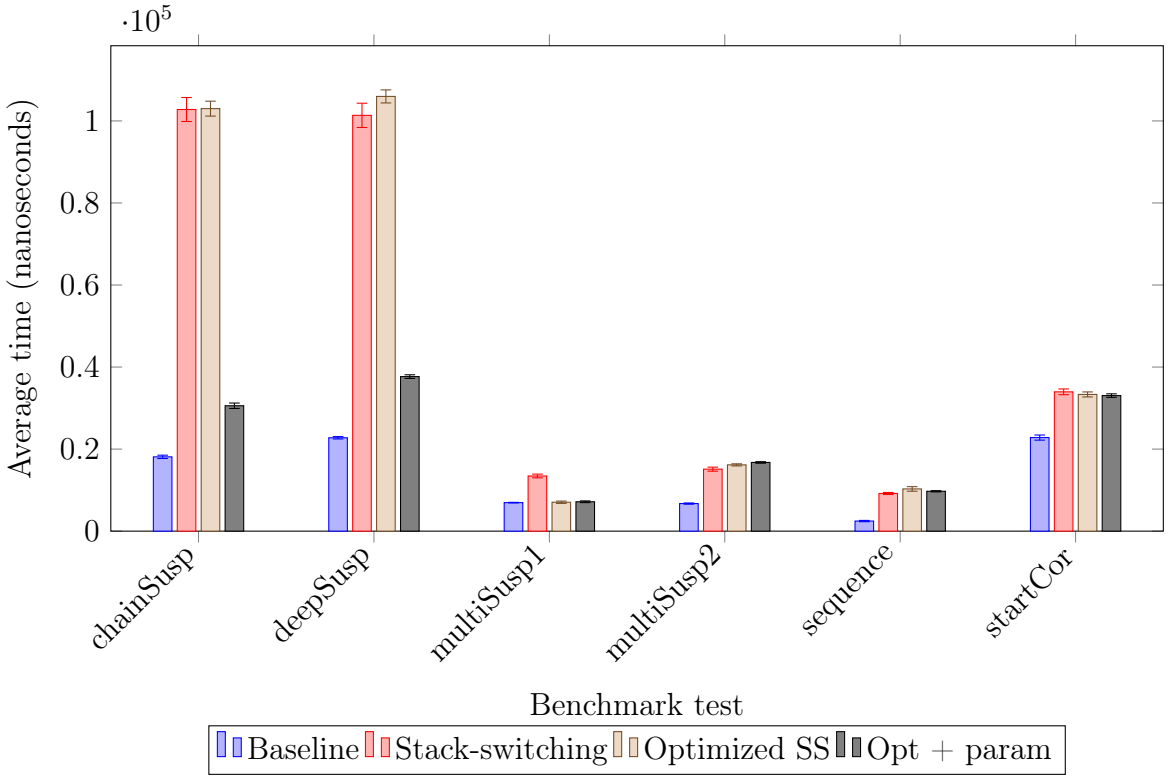


Figure 5.2: Benchmark comparison in dce mode, V8 runtime (error bars show 95% CI)

5.3.2 Reference Interpreter

The Reference Interpreter [20] is a reference interpreter with a proof-of-concept implementation of Stack-switching. It is not designed to be fast, and performance benchmarks on it are of limited interest as it is not used as a production VM. However, it is the VM that was initially used for testing, and at that time no production VMs were available to run benchmarks. The results are included here for completeness.

The following revisions were used:

- **Baseline**: commit `0dbc8c49`
- **Stack-switching**: commit `01ae026e`
- **Optimized SS**: commit `c6c7bc2e`

Raw results

Tables 5.6 and 5.7 show the raw benchmark results for dev and dce modes respectively.

Table 5.6: Benchmark results in dev mode (average time in milliseconds, Reference Interpreter)

Test	Baseline	Stack-switching	Optimized SS
chainSusp	576	372	657
deepSusp	690	425	743
multiSusp1	972	761	91
multiSusp2	74	98	146
sequence	29	44	66
startCor	255	376	519

Table 5.7: Benchmark results in dce mode (average time in milliseconds, Reference Interpreter)

Test	Baseline	Stack-switching	Optimized SS
chainSusp	519	297	554
deepSusp	601	330	629
multiSusp1	926	709	61
multiSusp2	48	60	93
sequence	18	26	40
startCor	152	218	310

Percentage Changes

Table 5.8 presents the percentage changes relative to the baseline. Positive values indicate performance regression (slower execution), while negative values indicate improvement (faster execution).

Table 5.8: Percentage change compared to baseline (Reference Interpreter)

Test	Mode	Baseline (ms)	Stack-switching	Optimized SS
chainSusp	dev	576	−35.5%	+14.1%
deepSusp	dev	690	−38.4%	+7.7%
multiSusp1	dev	972	−21.6%	−90.6%
multiSusp2	dev	74	+32.6%	+96.9%
sequence	dev	29	+51.0%	+124.4%
startCor	dev	255	+47.3%	+103.6%
chainSusp	dce	519	−42.9%	+6.7%
deepSusp	dce	601	−45.1%	+4.7%
multiSusp1	dce	926	−23.4%	−93.4%
multiSusp2	dce	48	+25.1%	+94.4%
sequence	dce	18	+44.7%	+119.5%
startCor	dce	152	+43.2%	+103.8%

Visual Comparison

Figures 5.3 and 5.4 provide a visual comparison of the benchmark results across all three implementations.

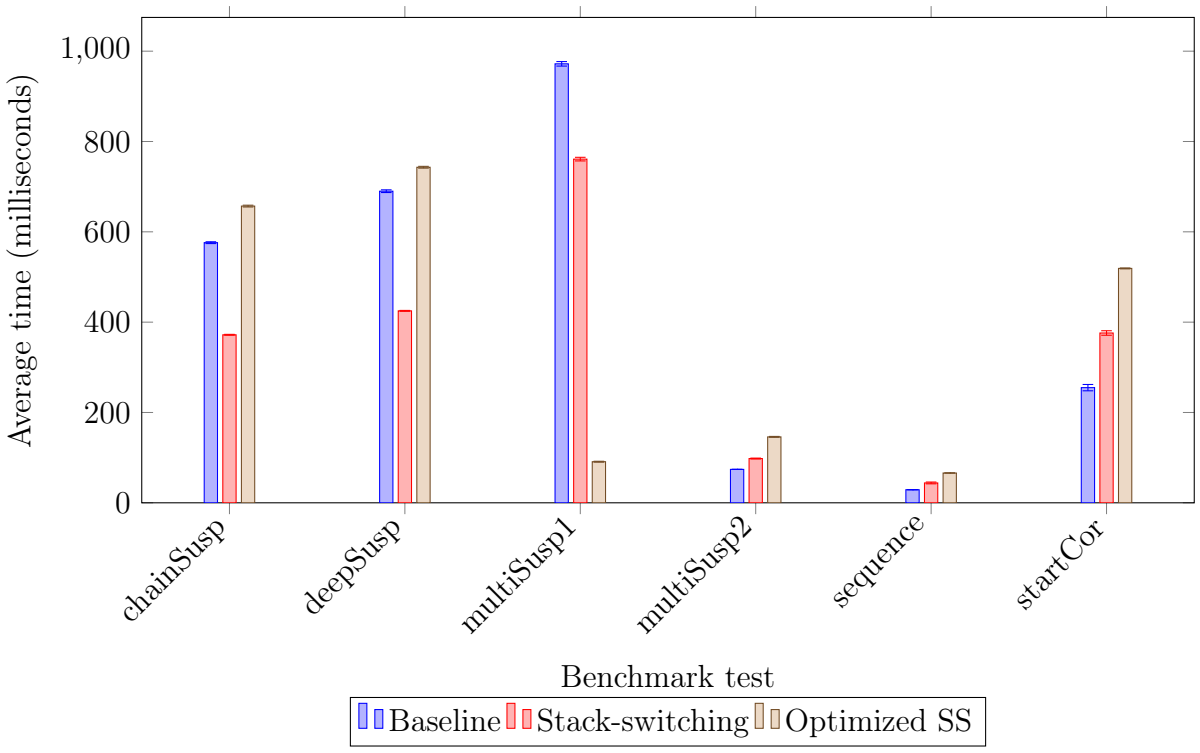


Figure 5.3: Benchmark comparison in dev mode, Reference Interpreter (error bars show 95% CI)

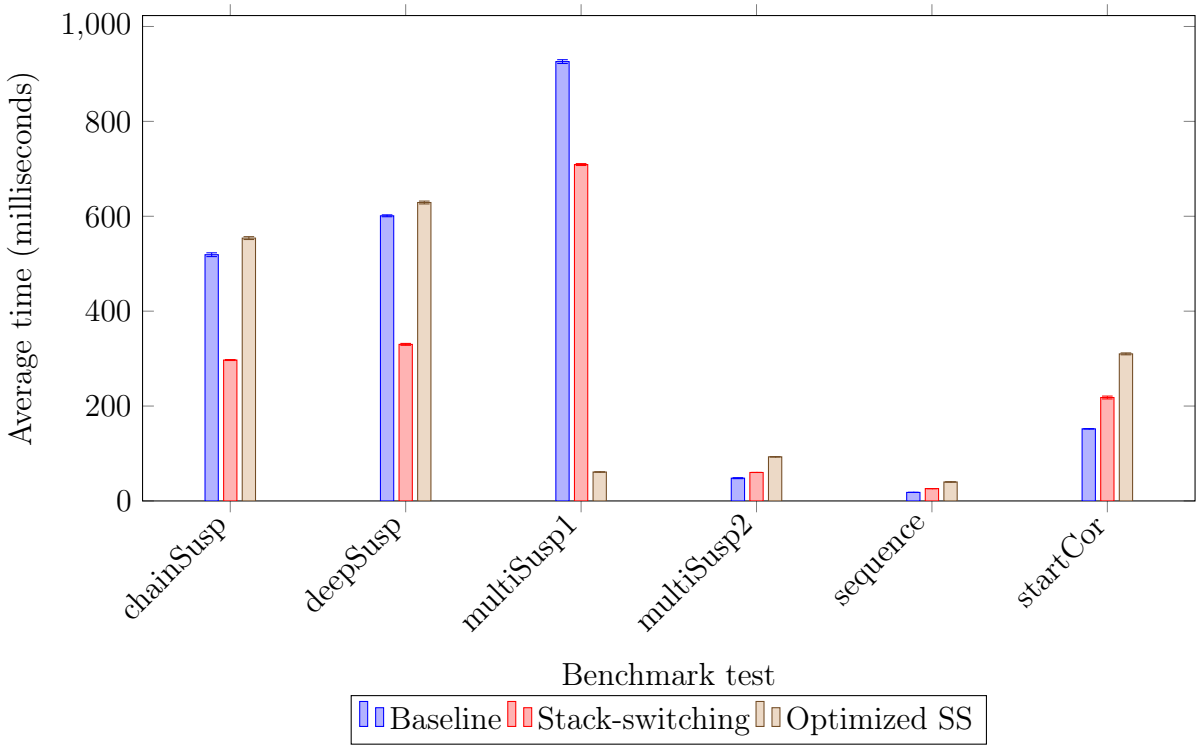


Figure 5.4: Benchmark comparison in dce mode, Reference Interpreter (error bars show 95% CI)

5.3.3 Analysis

The Stack-switching implementation is noticeably slower than the baseline on V8. The Reference Interpreter results are less severe, but the Reference Interpreter is not a production VM and does not reflect the experience of actual users.

`chainSuspensions` and `deepSuspensions` show the worst regression due to the nature of the V8 implementation, showing a 4.5–5.7 times slowdown. Passing the `--experimental-wasm-growable-stacks` flag reduces the regression to around 1.7 times compared to the baseline, though other cases become slightly worse. Whether this flag is beneficial depends on the particular application.

Among the presented benchmarks, `multipleSuspensions2` is the closest to what would appear in production code. This benchmark is around 2–2.5 times slower than the baseline on V8, which is a noticeable regression.

The `multipleSuspensions1` benchmark is comparable to the baseline, but the immediate resume pattern used in this benchmark is very unlikely to appear in real code. Resuming a continuation immediately inside the suspend lambda contradicts the purpose of suspending: if the resume value is available inside the suspend lambda, the function does not need to suspend at all. The immediate-resume optimization therefore has limited practical value.

Overall, the two implementation ideas presented in this thesis show very similar

performance results.

5.4 Summary

The Stack-switching implementation leads to a modest but noticeable decrease in binary size. The decrease is around 10% for a small application with little to no business logic written with Compose Multiplatform. The decrease would be smaller for an application that has business logic not involving Kotlin Coroutines, since the parts of the binary that do not use coroutines are not affected by the new implementation.

On the other hand, performance benchmarks show a noticeable regression. The cases most similar to production code are around 2–2.5 times slower. Cases with intensive suspension chaining are around 1.7 times slower if the `--experimental-wasm-growable-stacks` flag is passed to V8, and around 4.5–5.7 times slower without it. Cases with the immediate resume pattern are comparable to the baseline, but they are unlikely to appear in production code.

5.5 Threats to Validity

Several factors limit the generalizability of the evaluation results.

- **Limited number of applications.** The size benchmark uses a single small Compose Multiplatform project generated by the Kotlin Multiplatform Wizard. The result may not generalize to larger or more complex applications.
- **Immature V8 Stack-switching implementation.** The V8 Stack-switching support is still under active development. The observed performance regression may decrease as the implementation matures.
- **Low-level benchmarks.** The performance benchmarks focus on low-level coroutine primitives. They may not reflect the performance characteristics of higher-level application code.
- **Single VM.** Production benchmarks were only run on V8. Other production WebAssembly engines could not be tested due to incomplete or incompatible Stack-switching support.
- **Correctness test failures.** Six out of 523 coroutine tests fail due to an implementation bug (missing `Any?` casts). While this bug does not affect the core implementation, it indicates that the compiler needs some adjustments before the new implementation can be merged to master.

5.6 Note on other VMs

No benchmarks for other production VMs are included because those VMs lack features necessary for the Kotlin Coroutines implementation to function. The following is a list of VMs that have some progress in implementing the Stack-switching proposal, but were not tested due to various issues.

- **Wasmer**: according to the Wasm feature roadmap [4], Wasmer fully supports Stack-switching. Unfortunately, it doesn't support another Wasm feature that is necessary for Kotlin/Wasm as a whole: Garbage collection.
- **SpiderMonkey (Mozilla Firefox)**: Stack-switching is only partially implemented. The implemented feature subset is not enough to run the Kotlin Coroutines implementation [21, 8]. While adapting the implementation to run only on the supported primitives is technically possible, it is not straightforward and the result would be substantially different from the initial implementation.
- **Wasmtime**: Stack-switching implementation in Wasmtime has a bug which prevents Kotlin Coroutines from running properly [22].

One more VM that could technically be tested is Wizard [23]. It has support for Stack-switching, but like Reference Interpreter, it is not a production VM, so the benchmark results would be of limited interest.

Chapter 6

Conclusion

This thesis presented an alternative implementation of Kotlin Coroutines for Kotlin/Wasm based on the WebAssembly Stack-switching proposal. The implementation maps Kotlin coroutine primitives to Wasm Stack-switching instructions, replacing the state-machine transformation approach used by the existing Kotlin/JS-based implementation.

The evaluation showed measurable improvements in binary size: approximately 10.8% reduction in production builds and 2.3% in development builds for a small Compose Multiplatform project. On the other hand, performance benchmarks showed a noticeable regression. The benchmark cases most representative of real-world code are around 2–2.5 times slower than the baseline in V8. Cases with intensive suspension chaining regress by a factor of 4.5–5.7 without the `--experimental-wasm-growable-stacks` flag, and around 1.7 times with it.

Some of the observed performance regressions may decrease or even disappear as both the V8 Stack-switching implementation and the Kotlin compiler support for it are developed further.

Current state

The development of Kotlin Coroutines with Stack-switching is continued by JetBrains. At the moment of writing this, the implementation is being prepared for release as an experimental feature under a compiler flag. The production implementation is based on the second implementation idea, which uses the immediate resume optimization.

There are some benchmark results available for the production version of the implementation. For example, KotlinConfApp [24] was used for the size benchmark of production implementation. Compiling it with the Stack-switching implementation gave around a 5% improvement in binary size, which is consistent with the analysis of the size benchmark presented in this thesis. Low-level performance benchmarks on the production implementation also show a noticeable regression in performance, comparable to the previously presented performance benchmark results. There are also

higher-level Compose Multiplatform benchmarks available: those benchmarks show performance results comparable to the baseline, being slower on some test cases and faster on others. These results were presented at a meeting of the Wasm Community Group [25].

JetBrains is actively communicating with the V8 team regarding the observed regressions from the Stack-switching implementation [20]. The V8 team has indicated that their Stack-switching implementation is still in active development, and further performance improvements may follow.

Future work

Several directions for future work follow from this thesis. First, the current implementation allocates a new `WasmContinuation` object for each suspension point. Reusing or pooling these objects could reduce garbage collection pressure and improve performance. Second, a hybrid compilation strategy could be explored: using the state-machine approach for some functions and Stack-switching for others. Finally, once other production WebAssembly engines add sufficient support for the Stack-switching proposal, benchmarks should be repeated on those engines to determine whether the performance regression is specific to the current V8 implementation.

Bibliography

[1] WebAssembly Community Group. Stack-switching proposal: Generator example, 2024. URL https://github.com/WebAssembly/stack-switching/blob/main/proposals/stack-switching/examples/generator.wast. Accessed: 2026-05-15.

[2] WebAssembly Community Group. Webassembly specification, 2024. URL https://webassembly.github.io/spec/core/. Accessed: 2026-05-15.

[3] JetBrains. Compose multiplatform, 2024. URL https://www.jetbrains.com/compose-multiplatform/. Accessed: 2026-05-15.

[4] WebAssembly Community Group. Webassembly features, 2024. URL https://webassembly.org/features/. Accessed: 2026-05-15.

[5] Alon Zakai. Asyncify in practice, 2022. URL https://kripken.github.io/talks/2022/asyncify.html. Presentation at WebAssembly Stack Subgroup, January 2022. Accessed: 2026-05-19.

[6] WebAssembly Community Group. Js promise integration proposal, 2024. URL https://github.com/WebAssembly/js-promise-integration. Accessed: 2026-05-19.

[7] Thibaud Michaud. Jspi proposal, 2022. URL https://github.com/WebAssembly/meetings/blob/main/stack/2022/9-26-22.md. Presentation at WebAssembly Stack Subgroup, September 2022. Accessed: 2026-05-19.

[8] Mozilla. Bug 2023217 — wasm: Add stack-switching proposal and implement JSPI with it, 2026. URL https://phabricator.services.mozilla.com/D293529. Accessed: 2026-05-19.

[9] Sam Lindley and Daniel Hillerström. Wasmfx: Effect handlers for webassembly, 2023. URL https://wasmfx.dev/. Accessed: 2026-05-19.

[10] Sam Lindley. Wasm with typed continuations continued, 2021. URL https://github.com/WebAssembly/meetings/blob/main/stack/2021/sg-11-1.md. Presentation at WebAssembly Stack Subgroup, November 2021. Accessed: 2026-05-19.

[11] Sam Lindley. Wasmfx: Status update, 2023. URL https://github.com/WebAssembly/meetings/blob/main/stack/2023/presentations/wasmfx-stacks2023-07.pdf. Presentation at WebAssembly Stack Subgroup, July 2023. Accessed: 2026-05-19.

[12] Daniel Hillerström. Typed continuations, the wasmtime perspective, 2023. URL https://github.com/WebAssembly/meetings/blob/main/stack/2023/stack-2023-07-17.md. Presentation at WebAssembly Stack Subgroup, July 2023. Accessed: 2026-05-19.

[13] Cherry Mui, Jeremy Faller, Austin Clements, Michael Knyszek, and Michael Pratt. Go-lang on wasm, 2021. URL https://github.com/WebAssembly/meetings/blob/main/stack/2021/sg-7-12.md. Presentation at WebAssembly Stack Subgroup, July 2021. Accessed: 2026-05-19.

[14] Ayke van Laethem. Tinygo, 2021. URL https://github.com/WebAssembly/meetings/blob/main/stack/2021/sg-10-18.md. Presentation at WebAssembly Stack Subgroup, October 2021. Accessed: 2026-05-19.

[15] Anil Madhavapeddy. Multicore ocaml in practice, 2022. URL https://github.com/WebAssembly/meetings/blob/main/stack/2022/sg-2-7.md. Presentation at WebAssembly Stack Subgroup, February 2022. Accessed: 2026-05-19.

[16] Paul Schonfelder. Challenges in implementing erlang processing in wasm, 2020. URL https://github.com/WebAssembly/meetings/blob/main/stack/2020/SG-10-19.md. Presentation at WebAssembly Stack Subgroup, October 2020. Accessed: 2026-05-19.

[17] JetBrains. Kotlin multiplatform wizard, 2024. URL https://kmp.jetbrains.com/. Accessed: 2026-05-15.

[18] JetBrains. Kotlin, 2024. URL https://github.com/JetBrains/kotlin. Accessed: 2026-05-19.

[19] StatCounter. Browser market share worldwide, 2025. URL https://gs.statcounter.com/browser-market-share. Accessed: 2026-05-15.

[20] WebAssembly Community Group. Webassembly stack-switching proposal, 2024. URL https://github.com/WebAssembly/stack-switching. Accessed: 2026-05-15.

[21] Mozilla. Meta: Implement webassembly stack-switching in spidermonkey, 2025. URL https://bugzilla.mozilla.org/show_bug.cgi?id=2032477. Accessed: 2026-05-19.

[22] Bytecode Alliance. Stack switching: Wasmfuncref resume causes trap, 2025. URL https://github.com/bytecodealliance/wasmtime/issues/12941. Accessed: 2026-05-19.

[23] Ben L. Titzer. Wizard engine, 2025. URL https://github.com/titzer/wizard-engine. Accessed: 2026-05-19.

[24] JetBrains. Kotlinconf app, 2024. URL https://github.com/JetBrains/kotlinconf-app. Accessed: 2026-05-18.

[25] WebAssembly Community Group. Agenda for the may 18th video call of webassembly's stack subgroup, 2026. URL https://github.com/WebAssembly/meetings/blob/main/stack/2026/stack-2026-05-18.md. Accessed: 2026-05-18.

Appendix

Listing 1: Example from startCoroutineUninterceptedOrReturn.kt

```
suspend fun suspendHere(): String =
    suspendCoroutineUninterceptedOrReturn { x ->
     x.resume("OK")
     COROUTINE_SUSPENDED
}

suspend fun suspendWithException(): String =
    suspendCoroutineUninterceptedOrReturn { x ->
     x.resumeWithException(RuntimeException("OK"))
     COROUTINE_SUSPENDED
}

fun builder(shouldSuspend: Boolean, c: suspend () -> String): String {
     var fromSuspension: String? = null

     val result = try {
          c.startCoroutineUninterceptedOrReturn(object: Continuation<
             String> {
               override val context: CoroutineContext
                    get() =  EmptyCoroutineContext

               override fun resumeWith(value: Result<String>) {
                    fromSuspension = try {
                         value.getOrThrow()
                    } catch (exception: Throwable) {
                         "Exception: " + exception.message!!
                    }
               }
          })
     } catch (e: Exception) {
          "Exception: ${e.message}"
     }

     if (shouldSuspend) {
          if (result !== COROUTINE_SUSPENDED) throw RuntimeException("
             fail 1")
```

```
33          if (fromSuspension == null) throw RuntimeException("fail 2")
34          return fromSuspension!!
35      }
36
37      if (result === COROUTINE_SUSPENDED) throw RuntimeException("fail 3
            ")
38      return result as String
39  }
40
41  fun box(): String {
42      if (builder(false) { "OK" } != "OK") return "fail 4"
43      if (builder(true) { suspendHere() } != "OK") return "fail 5"
44      if (builder(true) { suspend{}(); suspendHere() } != "OK") return "
            fail 51"
45
46      if (builder(false) { throw RuntimeException("OK") } != "Exception:
             OK") return "fail 6"
47      if (builder(true) { suspendWithException() } != "Exception: OK")
            return "fail 7"
48      if (builder(true) { suspend{}(); suspendWithException() } != "
            Exception: OK") return "fail 71"
49
50      if (builder(true, ::suspendHere) != "OK") return "fail 8"
51      if (builder(true, ::suspendWithException) != "Exception: OK")
            return "fail 9"
52
53      return "OK"
54  }
```